15강

프로그래밍 언어론

객체지향과 다형성

컴퓨터과학과 김진욱교수

학습목차



1. 추상 자료형
2. 객체와 클래스
3. 다형성

01

추상 자료형

1. 추상 자료형

추상화 (abstraction)

- 복잡한 대상을 간략하게 나타내는 것

추상화 방법

- 추리기: 대상의 관심 있는 부분만 추려 나타냄
- 삭제하기: 대상의 관심 없는 부분을 삭제하여 나타냄

1. 추상 자료형

프로그래밍 언어의 추상화 종류

제어 추상화

- 복잡한 제어 과정을 단순하게 제공
- 예: 냉장고에 코끼리 넣기
  push(s, x): 스택 s에 어떤 자료 x 넣기

자료 추상화

- 복잡한 자료 구조를 단순하게 제공
- 예: 학생 ⇒ 학번, 이름, 수강과목, 성적 등(성적 처리 목적)
  원소 저장 공간, 스택 포인터 등 ⇒ 스택 자료 구조

1. 추상 자료형

❑ 프로그래밍 언어의 추상화 지원

제어 추상화

- 제어 구조, 서브프로그램으로 지원
  - 초기 프로그래밍 언어의 역점 사항
  - '어떻게'는 숨기고 '무엇이' 수행되는지 나타냄

```
sort(list, len);
```

1. 추상 자료형

프로그래밍 언어의 추상화 지원

자료 추상화

- 자료 구조, 추상 자료형으로 지원
  - 제어 추상화가 발달함에 따라 자료 구조에 연산을 적용
  - 자료와 연산을 한 묶음으로 다루게 되면서 추상 자료형이 발전함

```
s = Stack(int);
for (int n = 1; n < 5; n++)
  push(s, n);
while (!isEmpty(s))
  printf("%d\n", pop(s));
```

1. 추상 자료형

캡슐화와 정보 은닉

캡슐화(encapsulation)

- 여러 요소를 하나의 단위로 묶을 수 있는 기능
- 예: C 언어의 구조체, 공용체, 함수 등

```
struct person
{
  char name[20];
  int birth;
  double height;
  double weight;
};
```

```
union uni
{
  int i;
  char a;
  double f;
};
```

1. 추상 자료형

캡슐화와 정보 은닉

정보 은닉 (information hiding)

- 구현 세부 사항을 숨기고 인터페이스만 외부에 공개하는 기능
- 예: C 언어의 파일 범위 식별자
  C++의 클래스 내 전용 멤버나 보호 멤버

```
static int f;

int usingF()
{
   ...
}
```

```
class test{
  private:
    int a;
  protected:
    int b;
  public:
    int usingA() { ... }
};
```

1. 추상 자료형

- 캡슐화와 정보 은닉
  - 두 용어는 같은 의미로 사용되기도 함
  - 장점
    - 구현에 종속적인 코드 사용을 막을 수 있음
    - 파급 효과를 막을 수 있음

파급 효과(ripple effect, 도미노 효과)

- 어느 한 부분의 코드 변경이 예기치 못한 범위까지 연속적으로 코드 변경을 불러오는 현상

1. 추상 자료형

- 추상 자료형(ADT: Abstract Data Type)
  - 자료 집합과 적용 가능한 연산을 함께 선언한 자료형
  - 구현 세부 사항은 캡슐화를 통해 외부에 공개되지 않음
  - 해당 타입의 자료를 사용하려면 외부에 공개된 인터페이스를 사용해야만 함

```
structure queue =
  operations ENQ, DEQ, ISEMPTY
  representation
    ...
  end rep
  procedure ENQ(...)
    ...
  end ENQ
  procedure DEQ(...)
    ...
  end DEQ
  procedure ISEMPTY(...)
    ...
  end ISEMPTY
  procedure NEXT(...)
    ...
  end NEXT
  begin
    initialization code
  end
end queue
```

1. 추상 자료형

- 추상 자료형의 장점
  - 수정 용이성
    - 자료형을 사용하는 부분을 바꾸지 않고 구현 코드를 바꿀 수 있음
  - 재사용성
    - 여러 상황에서 같은 코드를 재사용할 수 있음
  - 보안성
    - 구현 세부 사항을 프로그램의 다른 곳에서 바꾸지 못하도록 보호함

1. 추상 자료형

- 자료 표현이 없는 추상 자료형
  - 추상화 수준을 최대한 높인 경우
  - 연산만 정의하고 있는 자료형
    - Java의 인터페이스(interface)
  - 자료형의 연산 사이에 만족해야 하는 법칙 존재

```
isEmpty(makeStack()) == True
isEmpty(push(s, e)) == False
pop(push(s, e)) == e
```

1. 추상 자료형

대수적 자료형

- 타입 자체에 적용할 수 있는 대수적 연산을 이용하여 만들 수 있는 새로운 타입
- 예: 데카르트 곱(Cartesian product)
  - 수학: 집합 A, 집합 B → 집합 A×B
  - 컴퓨터: 자료형 T1, 자료형 T2 → 자료형 (T1, T2)

02

객체와 클래스

2. 객체와 클래스

- 객체지향 개념
  - 시뮬레이션 분야에서 발전
    - 객체: 실세계의 구체적인 대상을 추상화하여 나타냄
  - GUI (Graphical User Interface)
    - 초기 GUI: 책상을 모형화
  - 추상 자료형 구현에 매우 효과적

2. 객체와 클래스

객체

- 특정 상태를 유지하며 외부 요구에 반응하는 행태를 보이는 대상
- 필요에 따라 상태가 바뀔 수 있음

용어

- 속성: 객체의 상태를 정의하는 값
- 필드: 속성을 저장하고 있는 이름
- 메소드: 객체에 대한 요청을 처리하는 서브프로그램
- 메시지: 요청 구문 전체를 객체에 전달하는 것

2. 객체와 클래스

클래스

- 같은 메소드 집합과 필드 항목을 공유하는 객체의 묶음
- 객체 생성의 틀

용어

- 클래스의 인스턴스
  - 해당 클래스를 통해 생성된 객체
- 메타클래스
  - 모든 클래스의 클래스 구실을 하는 객체

2. 객체와 클래스

타입과 클래스

- 정적 타입을 지원하는 객체지향 언어
  - 클래스가 타입이 됨
  - 객체는 해당 타입의 원소가 됨
  - 필드와 메소드의 개수나 이름 등이 수행 중에 바뀌지 않음
- 동적 타입을 지원하는 객체지향 언어
  - 필드와 메소드의 개수나 이름 등이 수행 중에 바뀔 수 있음
  - 오리 타입

오리 타입 (duck typing)

- 지원하는 메소드에 따라 타입이 결정되는 타입 시스템

2. 객체와 클래스

생성자와 소멸자

- 생성자
  - 객체를 생성하는 데 사용되는 특별한 메소드
  - 직접 호출하지 않고 new 연산자나
    다른 방법(객체 변수 선언 등)에 의해 자동으로 호출됨
  - 중복정의 가능: 필드 초기화를 위한 인수의 다양한 조합 지원

2. 객체와 클래스

생성자와 소멸자

- 소멸자
  - 객체가 소멸될 때 호출되는 특별한 메소드
  - 직접 호출하지 않고 delete 연산자나 영역 탈출 등에 의해 자동으로 호출됨
  - 중복정의되지 않음
  - 소멸자가 지원되지 않는 언어는 쓰레기 수집기(garbage collector) 사용

2. 객체와 클래스

C++ 클래스 작성 예

```
class stack {
  private:
    int *stack_ptr;
    int max_len;
    int top_ptr;

  public:
    stack(int size = 100) {
      stack_ptr = new int[size];
      max_len = size-1;
      top_ptr = -1;
    }

    ~stack() {
      delete[] stack_ptr;
    }

    void push(int number) {
      if (top_ptr == max_len)
        cout << "Error: stack is full\n";
      else
        stack_ptr[++top_ptr] = number;
    }

    int pop() {
      if (top_ptr == -1)
        cout << "Error: stack is empty\n";
      else {
        top_ptr--;
        return stack_ptr[top_ptr+1];
      }
    }

    bool isempty() {
      return (top_ptr == -1);
    }
};
```

2. 객체와 클래스

Java 클래스 작성 예

```
class Stack {
  private int[] stack_ref;
  private int max_len, top_index;

  public Stack(int size) {
    stack_ref = new int[size];
    max_len = size-1;
    top_index = -1;
  }

  public void push(int number) {
    if (top_index == max_len)
      System.out.println("Error: stack
is full");
    else
      stack_ref[++top_index] = number;
  }
```

```
  public int pop() {
    if (top_index == -1)
      System.out.println("Error: stack
is empty");
      return top_index;
    else {
      top_index--;
      return stack_ref[top_index+1];
    }
  }

  public boolean isempty() {
    return top_index == -1;
  }
}
```

2. 객체와 클래스

객체지향 언어 구현 방법

- 객체지향의 핵심
  - 메소드 호출이 수신 객체에 따라 다르게 동작할 수 있음
  - 호출 메소드의 늦은 바인딩이 일어나야 함
- 늦은 바인딩의 구현
  - 통상 가상함수 테이블(VTab)을 이용함

VTab

- 객체와 연관된 메소드의 코드 주소를 포함하고 있는 테이블
- 메소드가 호출되면 VTab의 해당 주소에 저장된 함수 호출

2. 객체와 클래스

객체지향 언어 구현 방법

- 객체지향의 핵심
  - 메소드 호출이 수신 객체에 따라 다르게 동작할 수 있음
  - 호출 메소드의 늦은 바인딩이 일어나야 함
- 늦은 바인딩의 구현
  - 통상 가상함수 테이블(VTab)을 이용함

C++

- `virtual` 키워드를 이용한 멤버 함수만 늦은 바인딩

- 늦은 바인딩은 다형성을 구현하는 한 가지 방법임

03

다형성

3. 다형성

- 다형성(polymorphism)
  - 여러 타입의 데이터를 같은 이름으로 처리할 수 있는 특성
  - 다형성 지원 수준에 따른 다형성 종류
    - 다형 서브프로그램
      - 여러 타입을 다룰 수 있는 서브프로그램
    - 다형 타입
      - 타입을 인수로 받는 타입

3. 다형성

- 서브프로그램의 다형성 종류
  - 경험적 다형성(ad-hoc polymorphism)
  - 매개변수적 다형성(parametric polymorphism)
  - 서브타입 다형성(subtype polymorphism)
    - 순수 다형성(pure polymorphism)이라고도 함

3. 다형성

- 경험적 다형성
  - 실제로 다른 서브프로그램에 같은 이름을 붙인 형태의 다형성
  - 사용자정의 서브프로그램 중복정의(overloading)
    - C++, Java, Ada 등 많은 객체지향 언어
  - 연산자 중복정의: 피연산자의 타입에 따라 다르게 연산자를 구현
    - 예: + – 정수형 덧셈 구현과 실수형 덧셈 구현은 다름
    - C++나 Python

3. 다형성

경험적 다형성

- 중복정의와 기본 인수는 충돌할 수 있음
  - 중복정의의 경우 인수 개수와 타입을 통해 호출 함수를 결정함
  - 인수 기본값이 주어졌을 때 호출 함수를 결정하는 방식과 충돌할 수 있음
  - 이런 경우에 시그니처 매칭 알고리즘 수행 중 오류가 발생함

시그니처 매칭 알고리즘

- 중복정의된 함수나 연산자의 실제 구현을 결정할 때 사용되는 알고리즘

```
void f(double b = 0.0);
void f();
f();
```

```
int g(int);
double g(int);
```

3. 다형성

매개변수적 다형성

- 타입을 매개변수로 전달받는 형태의 다형성
  - 타입 인수는 명시적으로 전달될 수도 있고 암시적으로 전달될 수도 있음

주요 언어의 예

- Ada: generic 패키지를 지원
- C++: 타입 인수를 전달하는 템플릿(template)
- Java: 타입 인수를 전달하는 제너릭

- Ada와 C++ 모두 범용 서브프로그램은 어떤 틀 구실을 하며, 실제 컴파일 시간에 구체적 타입에 대해 코드를 개별적으로 복사

3. 다형성

매개변수적 다형성

- 타입을 매개변수로 전달받는 형태의 다형성
  - 타입 인수는 명시적으로 전달될 수도 있고 암시적으로 전달될 수도 있음

```
template <typename Type>
void generic_sort(Type list[], int len) {
   ...
}
```

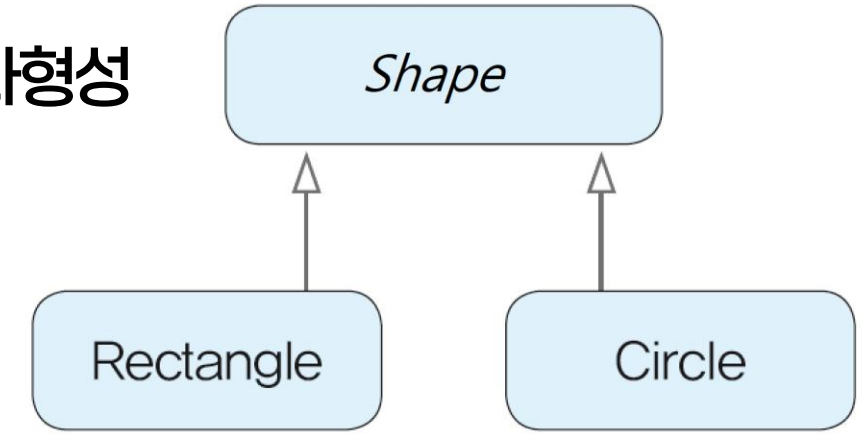
```
generic_sort<int>(int_list, 10);
generic_sort<double>(double_list, 20);
```

- Ada와 C++ 모두 범용 서브프로그램은 어떤 틀 구실을 하며,
  실제 컴파일 시간에 구체적 타입에 대해 코드를 개별적으로 복사

3. 다형성

순수 다형성

- 클래스 상속 관계(IS-A 관계)에 의해 결정되는 다형성
- 서브타입 다형성이라고도 함
- 적합 상속: IS-A 의미를 만족하는 상속



추상 클래스

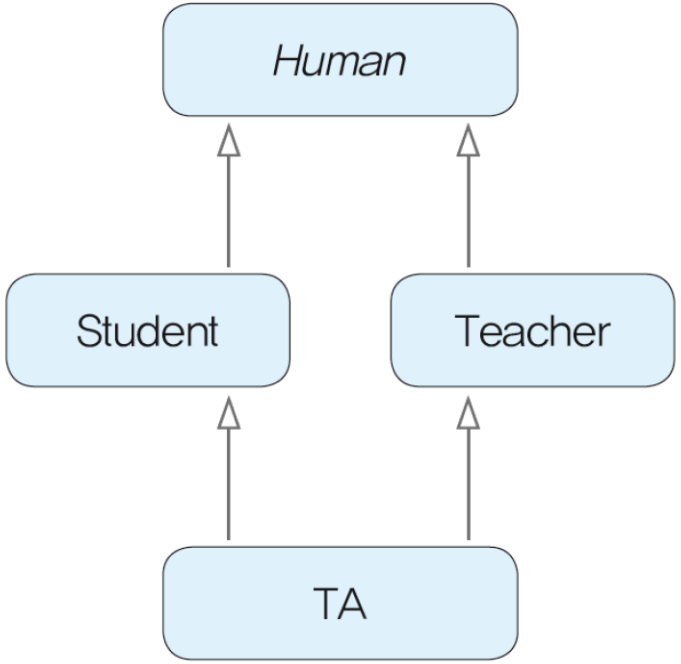
- 극도로 추상화되어 직접 객체를 생성할 수 없는 클래스

구상 클래스

- 직접 객체를 생성할 수 있는 클래스

3. 다형성

- 순수 다형성
  - 단일 상속
    - 하나의 부모 클래스로부터 상속이 이루어지는 경우
  - 다중 상속
    - 여러 부모 클래스로부터 상속이 이루어지는 경우
    - C++



다이아몬드 상속 문제

- 서브클래스에서 필드나 메소드를 호출할 때,
  다중 상속으로 인해 필드나 메소드 참조가 모호해지는 문제
- C++: 가상 클래스를 통한 가상 상속으로 해결

3. 다형성

인터페이스와 트레잇

인터페이스(interface)

- 추상 메소드 집합으로만 구성된 타입
- 구체적인 메소드 구현은 구상 클래스에 의해 구현됨
- 지원 언어: Java

```
interface Speakable {
    public void speak();
}

class Human implements Speakable {
    public void speak() {
        System.out.println("Hello? I'm a human");
    }
}
```

3. 다형성

인터페이스와 트레잇

트레잇(trait)

- 인터페이스와 유사하나, 일부 메소드는 구현되어 있을 수 있음
- 지원 언어
  - Java 8 이후 변경된 인터페이스는 트레잇의 특성을 반영
  - PHP, Rust 등
  - Ruby – 믹스인(mixin)이라는 이름으로 지원
  - Haskell – 클래스라는 이름으로 지원
- 트레잇은 상속된다거나 구현한다고 하지 않고 사용(use)한다고 함

```
trait Hello {
    public function sayHello() {
        echo 'Hello '.$this->getName();
    }
}

class Human {
    use Hello;
    public function getName() {
        return $this->name;
    }
}
```

정리하기

- 프로그래밍 언어에서 제공하는 추상화 종류는 제어 추상화와 자료 추상화로 나뉨
- 자료 집합과 적용 가능한 연산을 함께 선언한 타입을 추상 자료형이라고 함
- 객체란 특정 상태를 유지하며 외부 요구에 반응하는 행태를 보이는 대상을 추상화시킨 것임
- 객체에 따라 호출 메소드가 결정되도록 하기 위해서는 호출 메소드의 늦은 바인딩이 필요한데, 가상함수 테이블(VTab)로 늦은 바인딩을 구현함
- 다중 상속을 지원하면 다이아몬드 상속 문제가 발생함
- 객체를 직접 생성할 수 없을 정도로 극도로 추상화된 클래스를 추상 클래스라고 함
- 추상 클래스의 개념을 확장한 것으로 인터페이스와 트레잇을 들 수 있음

프로그래밍언어론

한 학기 동안
수고 많으셨습니다.